

StudyFlow Architecture

1. Project Overview

StudyFlow is a productivity web application that helps students manage goals, tasks, and progress.

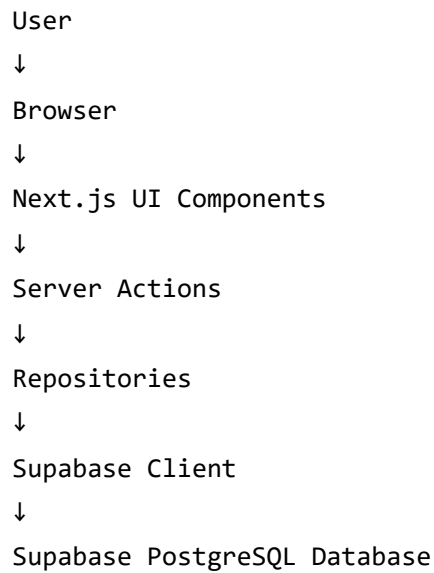
The app allows users to:

- Register and login
- Create, read, update, and delete goals
- Create, read, update, and delete tasks
- Link tasks to goals
- Create standalone daily tasks
- Mark tasks as Done / Todo
- Track progress using dashboard statistics

2. Tech Stack

Layer	Technology
Frontend	React
Framework	Next.js App Router
Language	JavaScript
Database	Supabase PostgreSQL
Auth	Supabase Auth
Styling	global.css
Deployment	Replit

3. High-Level Architecture



4. Application Layers

UI Layer

Responsible for displaying the interface and handling user interactions.

Examples:

Dashboard Page
GoalForm
GoalCard
TaskForm
TaskList
DashboardStats
EditGoalModal
EditTaskModal
Navbar

The UI layer should not directly contain database logic.

Actions Layer

Responsible for business logic and validation.

Examples:

```
createGoalAction()  
getGoalsAction()  
updateGoalAction()  
deleteGoalAction()  
  
createTaskAction()  
getTasksAction()  
updateTaskAction()  
deleteTaskAction()  
toggleTaskStatusAction()
```

Actions usually:

1. Receive input from the UI
2. Validate the input
3. Get the current user
4. Call a repository function
5. Return success or error

Repository Layer

Responsible only for database access.

Examples:

```
goalsRepo.js  
tasksRepo.js
```

Repository functions talk directly to Supabase:

```
createGoal()
getGoals()
updateGoal()
deleteGoal()

createTask()
getTasks()
getTasksByGoal()
getDailyTasks()
updateTask()
deleteTask()
```

Repositories should not handle UI logic.

Database Layer

Responsible for storing and protecting data.

Tables:

```
auth.users
public.goals
public.tasks
```

Security is handled using RLS policies.

5. Data Flow Examples

Create Goal Flow

User submits Goal form

↓

GoalForm

↓

handleCreateGoal()

↓

createGoalAction()

↓

goalsRepo.createGoal()

↓

Supabase

↓

goals table

↓

Goal created

↓

loadGoals()

↓

UI updates

Create Task Under Goal Flow

User submits Task form under a Goal

↓

TaskForm

↓

handleCreateTask(e, goalId)

↓

createTaskAction()

↓

tasksRepo.createTask()

↓

Supabase

↓

tasks table

↓

Task created with goal_id

↓

loadTasksForGoal(goalId)

↓

UI updates

Toggle Task Status Flow

```
User clicks checkbox
↓
TaskList
↓
handleToggleTaskStatus(taskId, nextStatus, goalId)
↓
toggleTaskStatusAction()
↓
tasksRepo.updateTask()
↓
Supabase
↓
tasks.status changes
↓
done_at updated or cleared
↓
loadTasksForGoal(goalId)
↓
loadAllTasks()
↓
UI updates
```

6. Authentication Architecture

Supabase Auth manages users.

Register

↓

Supabase Auth

↓

auth.users

Login

↓

Supabase Auth

↓

Session / Cookies

↓

Dashboard Access

The app uses:

- `lib/supabase/client.js` for browser/client components
- `lib/supabase/server.js` for server actions

Rule:

"use client" → use browser Supabase client

"use server" → use server Supabase client

7. Authorization

Authentication answers:

Who are you?

Authorization answers:

What are you allowed to access?

In this project:

A user can only access their own goals and tasks.

This is enforced using:

Row Level Security (RLS)

8. Folder Structure

```
studyflow/
|
├─ app/
|   │   └─ page.js
|   │   └─ layout.js
|   │   └─ globals.css
|   │   └─ login/
|   │       └─ page.js
|   │   └─ register/
|   │       └─ page.js
|   └─ dashboard/
|       └─ page.js
|
├─ actions/
|   └─ authActions.js
|   └─ goalActions.js
|   └─ taskActions.js
|
├─ repositories/
|   └─ goalsRepo.js
|   └─ tasksRepo.js
|
├─ lib/
|   └─ supabase/
|       └─ client.js
|       └─ server.js
|
├─ components/
|   └─ Navbar.jsx
|   └─ DashboardStats.jsx
|   └─ GoalForm.jsx
|   └─ GoalCard.jsx
|   └─ TaskForm.jsx
|   └─ TaskList.jsx
|   └─ EditGoalModal.jsx
|   └─ EditTaskModal.jsx
|
└─ package.json
```

9. Important Concepts Used

Concept	Meaning
MVP	Smallest useful version of the product
Architecture	How the project is structured
Data Flow	How data moves through the app
CRUD	Create, Read, Update, Delete
Repository Pattern	Separates database logic from UI/business logic
Server Actions	Backend functions inside Next.js
RLS	Database-level security
Props	Data/functions passed from parent component to child
State	Temporary UI data stored in React
Component Decomposition	Splitting large pages into smaller components
Aggregation	Calculating useful stats from data